

Q1 Git Config

Description

Objective:

Working with Git config is crucial for version control systems, as each Git commit utilizes this information

Git Config

Once you join a development project, you will be given access to the project repository. Which will be maintained in a Version Control System (VCS). There are lots of software used for it, here we will be learning about **GIT**.

You will be provided with a Git project URL and credentials to access it. The first thing you should do when you install Git is to set your [user name](#) and email address. This is important because every Git commit uses this information, and it's immutably baked into the commits you start creating.

In this activity, you will be adding the respective email and username from command line.

In command1.sh, .

Initialize GIT

Enter the below information as config

name as "**Programmer**"

email as "**Programmer@example.com**"

bio as "**I am a Git learner**"

company as "**Teknoturf**"

location as "**Chennai**"

Q2 Clone Repo

Description

Objective:

To work with Git: clone a repository into a new directory.

Clone Repo

Activity 1 (clone)

Developer A has started working on a shopping site development project, once a developer is allocated with the project, they will be given access to the project repo in Git. To start the development work, the developer will need a copy of the project code in their machine. It can be achieved using the clone command.

In **command.txt** enter the git commands to,

- 1) Initialize git
- 2) Clone the repository from the URL,

http://localhost:9080/student1/test1.git

UserName : student1

Password : studentpassword

Clone from "master" branch ONLY

Q3 Add, Commit And Push

Description

Objective:

To work with Git commands, you can add changes, commit, and push them into the repository.

Add, Commit and Push

Activity 3 (Add, Commit And Push)

You have cloned the repo and created a branch in the previous activities.... In continuation to that....

In this shopping site development project, Developer A has been allocated with a task to integrate the Employee management details into the existing options. Since it's a new feature development, all the code related to this feature should be added in Employee.java. Whenever we modify the file, it should be added into git.

Modified code should be checked-in into the Git repository regularly.

In Git, we will **commit** the code with a proper message. The message will have the short message of what code is being saved.

To check-in our code into the git, we will **push** all our code to remote repo after the commits.

NOTE :

You are in **DEV_feature** branch after previous activity

In **command.txt** enter the git commands to,

1) Add the below code to Employee.java,

```
public double averageTax(){  
    }  
}
```

2) Add Employee.java to GIT

3) Commit Employee.java using the comment "Method AverageTax included"

4) Push the changes in the **DEV_feature branch** to the **remote** repository

URL : <http://localhost:9080/student1/test1.git>

username : student1

password : studentpassword

Q4 Pull And Merge

Description

Objective:

To work with Git pull commands after cloning the repository, create a branch, and push the branch to the remote.

Pull and Merge

Activity 4 (Pull And Merge)

You have cloned the repo, created a branch and pushed the branch to remote in the previous activities.... In continuation to that....

Consider the scenario,

Developer A is working in his own branch (DEV_feature). In the meantime, other team members have worked on a new functionality to update the shopping site home page. And all their code changes are pushed into the a new branch **DEV_payment**.

Developer A will not have other branch changes. So he will need to **merge** the latest changes from the other branch with his branch to keep it up to date.

NOTE :

DEV_payment branch in remote is ALREADY created and it has the files.

In **command.txt** enter the git commands to,

1) Mention the command to display current branch name(this is to ensure that you are in **master branch**)

2) **Pull** the **DEV_payment** branch from remote origin

URL : http://localhost:9080/student1/test1.git

username : student1

password : studentpassword

3) Switch to **DEV_payment** branch using checkout (remote files must be added to local branch)

4) Switch to **DEV_feature** branch using checkout

5) Merge the **DEV_feature** branch with the **DEV_payment** branch

Q5 Merge – Resolve Conflict

Description

Objective:

To work with Git, a merge conflict occurs when Git is unable to automatically resolve differences in code between two commits.

Activity 5 (Conflict resolve)

Overview About Conflict :

Developer A has made some changes into existing files to handle a use case. Let's say, the changes are done in the file Payment.java, Developer A has added the payment instructions into the existing one.

In the same time Developer B has made some changes in the Payment.java file to improve the user experience in choosing the payment options.

Now both the developers have been working on the same file. Since they are working on different functionalities, they are not aware of the changes done by the other developer.

The Developer B has completed the work and it's been moved to the master branch after QA testing. Now to keep their own branch up to date with master. Developer A merges the master branch with their (DEV_feature) branch. Since both worked on the same file. The Developer A got conflict in the Payment.java file.

Now Developer A must resolve the conflicts by opening the file in which the conflict occurred and remove the conflict messages and commit the correct code into their (DEV_feature).

Exercise Scenario:

a) Employee.java is committed in 'master' with the below code.

```
public class Employee {  
  
}
```

b) A branch '**DEV_feature**' is created and a **checkout** to this branch is done.

c) Employee.java with below code is committed in this new branch

```
public class Employee {  
  
    int empid;  
    String name;  
  
}
```

d) A checkout to '**master**' is done with the below code in Employee.java.

```
public class Employee {  
    int empid;  
}
```

A MERGE CONFLICT WILL BE CREATED ONCE YOU MERGE THE BRANCH

Do the steps given below:

In **command1.txt** enter the git commands to,

- 1) Enter the command to display the current branch name (you are in 'master' branch now)
- 2) Merge with branch "**DEV_feature**"
- 3) Enter the command to display the difference in the Employee.java

NOTE:

At this point you must be getting **MERGE CONFLICT**. Observe the message in the console screen. To resolve the conflict,

In **command2.txt** enter the git commands to,

- 1) Modify Employee.java based on the 'diff' command output.
- 2) Add Employee.java to Git
- 3) Commit Employee.java using the comment "Employee is in sync with branch"
- 4) Merge the "DEV_feature" with master

Q6 Git Tags

Description

Objective:

To work with Git tags, consider them as specific points in Git history.

Git Tags

Git has the ability to tag specific points in a repository's history as being important. Tagging lets developers mark important checkpoints in the course of their projects' development.

2 Files are already given(Employee.java and Driver.java).

In **command1.sh** enter the git commands to,

- 1) Initialize git
- 2) Add ONLY Employee.java and Driver.java to git
- 3) Commit these 2 files using the comment "Initial commit Employee and Driver.java"

In **command2.sh** enter the git commands to,

- 4) Create a branch named "**DEV_feature**"
- 5) Switch to this new branch using **checkout** statement
- 6) Add the below code to Employee.java

```
public class Employee {  
    String name;  
    int id;  
  
    String designation;  
  
    double salary;  
  
    long phoneNo;  
}
```

- 7) Add Employee.java to Git
- 8) Commit Employee.java file using the comment "Employee with additional attributes"
- 9) Create a tag named **projectv1.1** with the tag message "**Project Starter Code**"
- 10) Create a new file(by clicking the "new" button).



Name the file as "EmployeeUtil.java".

Add the below code to "EmployeeUtil.java"

```
public class EmployeeUtil{  
    public void sortEmployee(List<Employee> empList){  
        }  
    }  
}
```

Click "Save All"

- 11) Add EmployeeUtil.java ONLY to Git
- 12) Commit the file EmployeeUtil.java using the comment "Added EmployeeUtil"
- 13) Create a tag named **projectv1.2** with the tag message "**Project With EmployeeUtil**"